

AnyLM Skill Building Guide

A technical guide to creating skills: instruction + tool bundles for local models

Version 1.0 · July 2026 · Covers the AnyLM desktop app and its NestJS auth backend

Contents: 1. What is a skill · 2. Architecture · 3. Runtime flow · 4. Custom skills (no code) · 5. Built-in connector skills (code) · 6. The OAuth connector lifecycle · 7. Writing skill instructions · 8. Security model · 9. Testing and troubleshooting · 10. Reference

1. What is a skill?

A **skill** is a named bundle of two things that are enabled and disabled as one unit: **instructions** injected into the system prompt, and **tools** the model may call. Where a plain tool answers 'what can the model do', a skill also answers 'when and how should it do it'. Enabling the Google Calendar skill, for example, gives the model both the `gcal_*` tools and standing guidance to check the calendar before proposing times.

AnyLM has two kinds of skills. **Built-in connector skills** ship with the app (Google Calendar, Outlook), talk to an external service over its REST API, and require the user to connect an account through OAuth. **Custom skills** are created in the Skills view by the user: they pair free-text instructions with any tools already defined in the Tools view (shell or HTTP).

Skills only apply while the  tools toggle is on in a chat. With tools off, no skill instructions are injected and no skill tools are offered.

2. Architecture

Layer	File	Responsibility
App (main)	<code>app/src/main/skills/builtins.js</code>	Built-in skill definitions: instructions plus runnable connector tools.
App (main)	<code>app/src/main/skills/registry.js</code>	Skill store (<code>userData/anylm-skills.json</code>), enable state, Ollama function schemas, system-prompt block.
App (main)	<code>app/src/main/skills/exec.js</code>	Executes connector tools: fetches a fresh OAuth token from the backend, runs the tool, confirms risky calls.
App (main)	<code>app/src/main/ipc.js</code>	<code>skills:*</code> IPC handlers; merges skill tools and instructions into the chat loop.
App (main)	<code>app/src/main/protocol.js</code>	<code>anylm://connectors/callback</code> deep link that completes the connect flow.
App (renderer)	<code>app/src/renderer/js/skills-view.js</code>	Skills manager UI: toggles, Connect/Disconnect, custom skill editor.
Backend	<code>auth-backend/src/connectors/providers.ts</code>	Provider table: OAuth endpoints, scopes, env-var names per connector.
Backend	<code>auth-backend/src/connectors/connectors.service.ts</code>	Consent URL, code exchange, token persistence, automatic refresh.
Backend	<code>auth-backend/src/connectors/connectors.controller.ts</code>	REST endpoints under <code>/connectors</code> .
Backend	prisma <code>ConnectorToken</code> model	Per user+provider OAuth tokens, unique on (userId, provider).

Tokens never touch the renderer. The Electron main process asks the backend for a live access token at execution time; the backend refreshes it server-side when it is near expiry.

3. Runtime flow

When a chat message is sent with tools enabled, ipc.js does four things:

1. `blocks.push(skillsRegistry.instructionsBlock())`
-> every enabled skill's instructions join the system prompt
2. `toolDefs = toolsRegistry.ollamaTools() + skillsRegistry.ollamaTools()`
-> skill tools are offered to the model (deduped by name)
3. model streams; tool_calls come back from Ollama
4. dispatch per call:
 - `skillsExec.owns(name)` -> `skills/exec.js` (connector tools)
 - otherwise -> `tools/exec.js` (shell / HTTP registry tools)

Connector execution in `skills/exec.js`: look up the tool on an enabled built-in skill, confirm with the user if the tool is risky, GET `/connectors/<provider>/token` from the backend, then await `tool.run(args, bearer)`. Whatever string `run()` returns is handed back to the model as the tool result, and the loop re-invokes the model (max 5 rounds).

Custom-skill tools are ordinary registry tools. Enabling a skill acts as the user's opt-in: the chat loop passes an allowlist so tools referenced by an enabled skill run even when globally switched off in the Tools view.

4. Building a custom skill (no code)

Open **Skills** → **New skill** and fill in four fields:

Field	What it does	Tips
Name	Label in the Skills list and in the system prompt.	Short and specific: 'Release notes', not 'Helper'.
Description	Shown in the Skills list only.	One sentence.
Instructions	Injected into the system prompt whenever the skill is enabled and tools are on.	Write to the model, not the user. State when to act, which tool to use, and what to confirm first.
Tools	Checkbox list of every tool in the Tools view.	Create the tools first (shell or HTTP), then bundle them.

Worked example: a 'Project notes' skill

1. In **Tools** → **New tool**, create a shell tool `save_note` with a command like `echo {text} >> ~/notes/inbox.md` and one required parameter `text`. 2. In **Skills** → **New skill**, name it 'Project notes', check `save_note`, and give it instructions:

When the user asks you to remember or file something, rewrite it as a one-line note (date first), then call `save_note` with it. Show the user the final line before saving.

3. Enable the skill and turn on ☒ in a chat. The model now knows the tool exists, when to reach for it, and what format to use.

5. Building a built-in connector skill (code)

Built-in skills live in code because their tools need real logic. Adding one takes two files: a provider entry on the backend and a skill object in the app. Everything else — the Skills UI row, the Connect button, token storage and refresh — lights up automatically. The example below adds a GitHub skill that lists and creates issues.

Step 1 — declare the provider (backend)

Add an entry to `auth-backend/src/connectors/providers.ts`:

```
github: {
  id: "github",
  label: "GitHub",
  authUrl: "https://github.com/login/oauth/authorize",
  tokenUrl: "https://github.com/login/oauth/access_token",
  scope: "repo",
  clientIdEnv: "GITHUB_CLIENT_ID",          // reuse existing OAuth app creds
  clientSecretEnv: "GITHUB_CLIENT_SECRET",
  extraAuthParams: {},
},
```

Register the callback URL with the provider:

`http://127.0.0.1:<PORT>/connectors/github/callback`. Set the two env vars in the backend's `.env`. Request the smallest scope that works.

Step 2 — define the skill (app)

Add a skill object to `app/src/main/skills/builtins.js` and include it in `BUILTIN_SKILLS`. Each tool is `{ name, description, risky, params, run(args, bearer) }`; `run()` must return a string for the model.

```
const github = {
  id: "github", // must match the provider id
  name: "GitHub",
  builtin: true,
  connector: "github", // ties Connect/token flow to the provider
  description: "List and create issues in the user's repositories.",
  instructions:
    "You can access GitHub with the gh_* tools. Always confirm the exact " +
    "repository (owner/repo) with the user before creating an issue.",
  tools: [
    {
      name: "gh_list_issues",
      description: "List open issues in a repository.",
      risky: false, // read-only -> no confirmation prompt
      params: [
        p("repo", "Repository as owner/name", true),
        p("max_results", "Max issues to return (default 10)"),
      ],
      run: async (args, bearer) => {
        const u = new URL(`https://api.github.com/repos/${args.repo}/issues`);
        u.searchParams.set("per_page", String(args.max_results || 10));
        return api(u.toString(), bearer); // shared fetch helper in builtins.js
      },
    },
    {
      name: "gh_create_issue",
      description: "Create an issue in a repository.",
      risky: true, // writes -> user must approve each call
      params: [
        p("repo", "Repository as owner/name", true),
        p("title", "Issue title", true),
        p("body", "Issue body"),
      ],
      run: async (args, bearer) =>
        api(`https://api.github.com/repos/${args.repo}/issues`, bearer, {
          method: "POST",
          body: JSON.stringify({ title: args.title, body: args.body || "" }),
        }),
    },
  ],
};

module.exports = { BUILTIN_SKILLS: [googleCalendar, outlook, github] };
```

Step 3 — there is no step 3

Restart both processes. The Skills view lists GitHub with a Connect button (or 'Not configured' until the env vars are set); once connected and enabled, the tools and instructions flow into every tools-on chat.

Design rules for connector tools

- Mark every write (create, send, delete) `risky: true` — the user then approves each call.
- All parameters are strings; parse inside `run()`.
- Return errors as strings starting with 'Error:' so the model can recover.
- Clip output (the shared `api()` helper truncates at 20,000 chars).
- Prefer query parameters like `max_results` with sane defaults so responses stay small for local models.

6. The OAuth connector lifecycle

Endpoint	Auth	Purpose
GET /connectors	JWT	Provider list with configured / connected / account flags (drives the Skills view).
POST /connectors/:p/start	JWT	Returns the consent URL; a random state maps the callback to the user (10-minute TTL, in-memory).
GET /connectors/:p/callback	state	Exchanges the code, upserts ConnectorToken, shows a success page, deep-links anylm://connectors/callback.
GET /connectors/:p/token	JWT	Returns a live access token; refreshes server-side when within 60 s of expiry.
DELETE /connectors/:p	JWT	Disconnect: deletes the stored token row.

The connect flow end to end: the Skills view calls skills:connect → main asks the backend for the consent URL → system browser opens → user approves → provider redirects to the backend callback → backend stores tokens and serves a page that deep-links back to the app → protocol.js resolves waitForConnector() → the Skills view refreshes.

Tokens live in the **ConnectorToken** table, unique per (userId, provider), with accessToken, refreshToken, expiresAt, scope, and accountEmail. Google only issues a refresh token when the consent URL carries access_type=offline and prompt=consent — the provider table handles that via extraAuthParams. Microsoft requires the offline_access scope for the same purpose.

Applying the schema change

```
cd auth-backend
bunx prisma migrate dev      # applies 20260703140000_connector_tokens
bunx prisma generate         # regenerate the client (required once)
```

7. Writing skill instructions

Instructions are the half of a skill that tools cannot express. They are injected as 'Skill "<name>": ...' inside an 'Enabled skills:' block of the system prompt. Local models follow short, imperative guidance best:

- **Say when to act:** 'When the user asks about their schedule, call gcal_list_events instead of guessing.'
- **Pin formats:** 'Dates must be RFC3339.' Local models hallucinate date formats more than anything else.
- **Order operations:** 'Read before you write: check the calendar before proposing times.'
- **Require confirmation** for irreversible actions: 'Show the user a draft before sending mail.'
- **Stay short** — every enabled skill consumes context on every message; two to four sentences is the sweet spot.

8. Security model

- **Risky confirmation:** any tool with `risky:true` routes through the same confirm dialog as shell tools; unanswered prompts auto-deny after 2 minutes.
- **Token custody:** OAuth tokens are stored only in the backend database; the app receives short-lived access tokens at call time and the renderer never sees them.
- **Scopes:** request the minimum per provider; scopes are fixed in `providers.ts`, not user-editable.
- **Opt-in twice:** a connector skill does nothing until the user both connects an account and enables the skill; per-chat, tools stay off until  is toggled.
- **Governance:** skill tool calls run inside the normal chat pipeline, so org policies (content filters, PII redaction, limits) still apply to what the model sends and receives.

9. Testing and troubleshooting

Symptom	Likely cause	Fix
'Not configured' on the Connect button	Missing client ID/secret env vars.	Set the provider's env vars in <code>auth-backend/.env</code> and restart the backend.
Connect opens the browser but never completes	<code>anylm://</code> scheme not registered (dev) or state expired.	Launch the app once so it claims the scheme; retry within 10 minutes.
'token expired and no refresh token is stored'	Provider issued no refresh token.	Reconnect the skill; for Google, offline access + forced consent must be present (they are, via <code>extraAuthParams</code>).
Model never calls the skill tools	Tools toggle off, skill disabled, or the model lacks function calling.	Turn on  , enable the skill, use a tool-capable model (e.g. llama3.1+).
Tool returns Error: HTTP 401/403	Scope missing or consent revoked.	Check scopes in <code>providers.ts</code> ; disconnect and reconnect.
Prisma error: <code>connectorToken</code> undefined	Client not regenerated after the migration.	Run <code>bunx prisma migrate dev && bunx prisma generate</code> .

Quick manual test loop: enable one skill, ask something that requires a read tool ('what's on my calendar tomorrow?'), verify the tool chip appears in the chat, then test one risky tool and confirm the approval dialog fires.

10. Reference

Custom skill storage (`userData/anylm-skills.json`)

```
{
  "enabledBuiltins": ["google-calendar"],
  "custom": [
    {
      "id": "s_lzx3...",
      "name": "Project notes",
      "description": "File notes to the inbox",
      "instructions": "When the user asks you to remember...",
      "toolNames": ["save_note"],
      "enabled": true
    }
  ]
}
```

```
}  
  ]  
}
```

IPC surface (window.api)

Call	Purpose
skillsList()	All skills with enabled state and tool names.
skillsToggle(id, enabled)	Enable/disable a skill.
skillsSave(skill) / skillsDelete(id)	Create, update, or remove a custom skill.
skillsConnectors()	Connector status list from GET /connectors.
skillsConnect(provider)	Run the full browser OAuth flow; resolves when connected.
skillsDisconnect(provider)	Delete the stored connector token.

Built-in skills shipped today

Skill	Connector	Tools
Google Calendar	google-calendar	gcal_list_events · gcal_create_event (risky)
Outlook	outlook	outlook_list_events · outlook_create_event (risky) · outlook_list_mail · outlook_send_mail (risky)

For plain tools (shell and HTTP, no OAuth), see the companion document *AnyLM Tool Building Guide*.